校易淘 · 校园二手交易系统

系 统 测 试 报 告

测试日期：2026 年 10 月 26 日
被测版本：main 分支 87c6aa7（含本轮缺陷修复）
测试环境：Windows 10.0.26200 / JDK 27 / MySQL 26.7 / Redis 7 / Node 24.21
报告结论：**通过**（44 + 19 + 34 项用例全部通过，发现 6 项缺陷并已修复复测）

一、报告摘要

本次测试对系统做了七个层面的验证：后端单元与集成、前端单元、接口功能、端到端业务闭环、角色权限、性能与限流、安全配置。测试不是"点一遍看看能不能用"，而是每个层面都设了明确的预期结果和记录实际输出。

1.1 结论

测试层面	用例/项	结果	结论
后端单元与集成	44	44 通过	交易状态机、并发、幂等、限流、支付、可观测性全部符合预期
前端单元	19	19 通过	HTTP 层、错误契约、令牌刷新、自动登出符合预期
接口功能	34	34 通过	鉴权、隔离、越权、边界、错误码、CORS、注入探针符合预期
端到端业务闭环	11 步	通过	下单→支付→发货→收货全链路可重复执行
角色权限	10 断言	通过	匿名 401、越权 404、数据按角色隔离
性能与限流	2 项	通过	1048 QPS、错误率 0%；登录限流第 11 次触发 429
安全配置	6 项	修复后通过	发现 6 项问题，全部修复并复测

> **一句话结论**：功能与工程质量达标；本轮测试发现并修复了 6 项缺陷，其中 2 项属安全类（Druid 控制台匿名可访问、JWT 密钥硬编码默认值），修复后已复测通过。剩余风险见第十四章。

1.2 关键数字

指标	实测值
后端用例通过率	44 / 44 = 100%（0 失败 0 错误 0 跳过）
前端用例通过率	19 / 19 = 100%
接口功能通过率	34 / 34 = 100%
业务模块行覆盖率（JaCoCo）	42.2%（930 / 2206 行）
压测吞吐（20 并发 / 10 秒）	1048.1 QPS，请求 10481，错误率 0.00%
压测延迟	avg 19.0ms，p95 34.0ms，p99 46.1ms
前端生产构建	1423 模块，dist 292.1 KB，JS gzip 81.34 KB
持续集成	CI #47 三作业全绿（后端 / 前端 / 容器编排）

二、测试范围与方法

2.1 范围

范围	内容	不在范围
功能	认证、商品、购物车、下单、支付、发货、收货、退款、收藏、地址	管理后台全部菜单
安全	认证与授权、越权、注入、CORS、敏感信息暴露、安全响应头	第三方渗透测试
性能	读接口吞吐与延迟、登录限流	长时间稳定性（Soak）
兼容	Chromium 内核浏览器、Windows 本机	移动端真机矩阵

2.2 方法

- **自动化用例**：后端 JUnit 5 + Spring Boot Test + Mockito，前端 node:test；由 CI 每次提交自动执行。
- **脚本化接口测试**：`scripts\test-api.ps1` 对每类风险各打一枪，固定记录 HTTP 状态码与响应体，输出 PASS/FAIL 汇总。
- **端到端演示**：`scripts\demo-business-loop.ps1`、`scripts\demo-rbac.ps1` 真实调用接口，不使用 mock、不直接改库。
- **数据复位**：演示前用 `scripts\reset-demo-data.ps1` 复位，保证结论可重复。
- **压测**：`scripts\loadtest.mjs`，按权重混合访问健康、列表、详情、类目四类读接口。
- **证据留存**：覆盖率报告、CI 运行记录、演示输出截图均随仓库保存。

2.3 通过判定

类型	判定标准
功能用例	实际结果与预期一致；断言失败即判不通过
接口状态码	与约定一致（如未授权 401、不存在 404、方法不支持 405、参数错误 400）
响应契约	/api/v1 统一为 {code, message, data}，不出现 problem+json
性能	错误率 0%，p95 在可接受范围
安全	不出现匿名可访问的敏感端点、不反射任意来源、不硬编码密钥

三、测试环境

项目	版本 / 配置
操作系统	Microsoft Windows NT 10.0.26200
JDK	27（编译目标 Java 17）
Maven	Apache Maven 3.9.9
MySQL	26.7.0，库 shop（45 张表），测试库 shop_test
Redis	7.x，127.0.0.1:6379
Node / npm	v24.21.0 / 11.19.0
前端构建	Vue 3.5 + Vite 6.4.1
Shell	PowerShell 5.1.26100
后端运行	ruoyi-admin.jar，端口 8080，健康检查 200

> 环境口径说明：JDK 27 是运行时版本，代码按 Java 17 编译，因此不存在"只有新 JDK 才能跑"的兼容问题；CI 在 Ubuntu + JDK 17 上同样通过，可作为跨环境的独立佐证。

四、测试结果总览

用例集	用例数	通过	失败	耗时 / 备注
后端单元与集成（11 类）	44	44	0	约 33 秒；含 1 个 12 秒的启动类用例
前端单元（2 文件）	19	19	0	136ms
接口功能（脚本）	34	34	0	4.6 秒
业务流程闭环	11 步	11	0	可反复执行
角色权限	10 断言	10	0	含越权 404
性能压测	1	1	0	20 并发 / 10 秒
限流验证	1	1	0	第 11 次 429

合计通过 44 + 19 + 34 = 97 个自动化用例，另有 22 项端到端与专项检查，全部通过。

五、后端单元与集成测试

后端用例全部落在 `ruoyi-admin/src/test/java/com/ruoyi/campus`，使用独立的 `shop_test` 库，不污染开发库。

测试类	用例	耗时	验证内容
StockConcurrencyTest	2	0.30s	库存 1 / 20 并发只成功 1 单；库存 10 / 20 并发恰好成功 10 单
IdempotencyConcurrencyTest	2	0.28s	同一幂等键并发 10 次只产生 1 单；顺序重复提交返回原批次
OrderLifecycleTest	5	0.33s	完整链路、取消回滚、退款拒绝回滚、退款完成回库、越权防护
PaymentGatewayTest	5	0.15s	预下单凭据、回调验签、失败分支、注册表分发、网关可扩展
RefreshTokenFlowTest	4	0.05s	一次性刷新令牌、用后失效、每用户唯一、畸形令牌拒绝
ObservabilityTest	10	0.21s	TraceId 生成与落 MDC、日志清洗、限流窗口/隔离/恢复/关闭
RedisSharedRateLimitTest	3	0.11s	跨实例共享配额、内存实例不共享、键前缀与 TTL
ApiContractTest	4	0.62s	OpenAPI 契约无幽灵端点、每个端点有摘要、前端调用与契约一致
ApiNotFoundHandlerTest	3	0.01s	/api/v1 未知路径返回真 404；其它路径维持原有行为
ApiMvcExceptionAdviceTest	5	0.50s	405、参数类型错误 400、校验失败与非法 JSON 统一响应体、非 API 路径回退
CampusHealthMetricsTest	1	12.0s	自定义健康指标注册（含真实上下文启动）

> **重点说明**：并发类用例不是"跑一遍看看有没有报错"，而是断言精确数量——库存 10 时并发 20 单必须恰好成功 10 单，多一单或少一单都算失败。

5.1 覆盖率

范围	指令	分支	行	方法
ruoyi-campus 业务模块	40.2%	32.9%	42.2%	45.5%

覆盖率由 `scripts\coverage.ps1` 生成 JaCoCo 报告；脚本会把源码复制到英文临时目录再运行，规避中文路径导致 agent 静默不产出的问题。门禁阈值 35%（行），低于则构建失败。

六、前端单元测试

文件	用例	验证内容
tests/http.test.js	15	查询参数编码、Bearer 注入、缺令牌拒绝、业务错误码保留、HTTP 200 内失败码、HTML 错误页识别、超时、网络失败不泄露内部信息、204 返回 null、X-Trace-Id、401 自动刷新与重放、并发 401 只刷新一次、刷新失败回调、无刷新令牌判定过期、非 401 不刷新
tests/contract.test.js	4	前端请求解析器自检、前端调用接口都存在于契约、每个操作有中文摘要、每个操作标注安全方案

```
ℹ tests 19
ℹ pass 19
ℹ fail 0
```

6.1 生产构建

```
vite v6.4.1 building for production...
✓ 1423 modules transformed.
dist/index.html                   0.51 kB │ gzip:   0.39 kB
dist/assets/index-CawuzhDJ.css   72.58 kB │ gzip:  12.66 kB
dist/assets/index-BpN2dh73.js   226.02 kB │ gzip:  81.34 kB
✓ built in 3.98s
```

七、接口功能测试

共 34 项，按风险分组。**本轮因为发现缺陷并修复，最终结果全部通过**；下表同时给出修复前的实际表现，便于追溯。

7.1 健康与探针（4 项）

用例	预期	结果
GET /api/v1/system/health	200	通过
GET /actuator/health	200	通过
GET /actuator/prometheus	200	通过
GET /druid/index.html（控制台默认关闭）	404	通过（修复后）

7.2 认证与令牌（7 项）

用例	预期	结果
错误密码登录	401	通过
不存在账号登录	401	通过
空手机号/密码登录	400	通过
无令牌访问订单	401	通过
伪造令牌访问订单	401	通过
非法刷新令牌	401	通过
当前用户 /user/info	200	通过

7.3 商品与分页边界（7 项）

用例	预期	结果
默认分页	200	通过
pageSize=1	200	通过
超大 pageSize（99999）	200（钳制）	通过，实际返回 size=12
非法 page=0	200（回落）	通过
商品详情不存在	404	通过
关键字搜索	200	通过
分类列表	200	通过

7.4 数据隔离与越权（4 项）

用例	预期	结果
买家「我的订单」	200	通过
买家「我卖的」为空	200	通过
不存在的订单详情	404	通过
买家调用发货（越权）	404	通过（不泄露资源存在性）

7.5 404 / 405 / 参数校验（5 项）

用例	预期	结果
未知接口	404	通过
已知接口用错方法 DELETE	405	通过（修复前为 200 + code 500）
下单缺必填字段	400	通过（修复前为 problem+json）
购物车加购非法数量	400	通过
购物车加购不存在商品	409	通过（业务冲突语义）

7.6 安全响应头与 CORS（4 项）

用例	实际	结果
X-Content-Type-Options	nosniff	通过
X-Frame-Options	SAMEORIGIN	通过
X-XSS-Protection	0	通过
未知 Origin 跨域	不返回 Access-Control-Allow-Origin	通过（修复前会反射任意 Origin）

7.7 注入探针（3 项）

用例	预期	结果
XSS 关键字（<script>）	不 500	通过，返回 0 条
SQL 注入关键字（' OR 1=1 --）	不 500	通过，返回 0 条
超大数字 ID	400	通过（修复前为 200 + code 500）

八、端到端测试

8.1 业务闭环（11 步）

```
[1] 买家登录（13800001001）                    OK
[2] 浏览商品列表（分页 20）                    OK  共 8 件在售
[3] 挑一件非本人出售、有库存的商品             OK  商品 #2 单价 680.00 库存 1
[4] 加入购物车                                 OK  合并同商品
[5] 提交订单（带幂等键）                       OK  状态 待付款
[6] 同一幂等键再提交（模拟连点两下）           OK  返回同一分组，幂等生效
[7] 发起支付（沙箱网关）                       OK  预下单数据
[8] 确认支付                                   OK  状态 -> 待发货
[9] 卖家登录并发货                             OK  状态 -> 待收货
[10] 买家确认收货                              OK  状态 -> 已完成
[11] 回看商品库存                              OK  1 -> 0
```

> 闭环里最关键的两点被真实覆盖：**幂等**（连点两下只生成一单）与**库存**（下单即原子扣减并置售罄）。演示前先跑 reset-demo-data.ps1，因此可反复执行，不会出现"第二次演示没有可下单商品"。

8.2 角色权限（10 断言）

请求	身份	结果	含义
GET /api/v1/orders	匿名	401	没有凭证直接拒绝
POST /api/v1/orders	匿名	401	写接口同样受保护
GET /api/v1/orders/sold	匿名	401	卖家视角同样受保护
GET /api/v1/orders	买家	200	只返回自己作为买家的订单
GET /api/v1/orders/sold	买家	200	空数组——没有卖出过东西
GET /api/v1/orders/sold	卖家	200	返回自己作为卖家的订单
PUT /orders/{no}/ship	买家（越权）	404	按"订单不存在"处理，不泄露资源存在性
PUT /orders/{no}/ship	卖家（合法）	200	状态机允许，发货成功
GET /actuator/health	匿名	200	探针必须匿名，否则负载均衡无法判断存活
GET /actuator/prometheus	匿名	200	仅暴露指标，不暴露业务数据

九、性能与限流

9.1 读接口压测

场景：20 并发持续 10 秒，按权重混合访问健康探活、商品列表、商品详情、类目列表。

```
并发 20 · 持续 10s · 请求总数 10481
总 QPS: 1048.1   成功 10481/10481   错误率 0.00%
RT  avg 19.0ms  p50 18.0ms  p95 34.0ms  p99 46.1ms  max 73.4ms

| 场景              | 请求数 | QPS   | avg(ms) | p95(ms) | p99(ms) | 错误率 |
| 健康探活          | 1163   | 116.3 | 4.1     | 6.5     | 11.0    | 0.00%  |
| 商品详情          | 2330   | 233.0 | 15.1    | 22.2    | 32.5    | 0.00%  |
| 商品列表(分页 20) | 4659   | 465.9 | 26.8    | 38.3    | 55.0    | 0.00%  |
| 类目列表          | 2329   | 232.9 | 14.9    | 21.9    | 31.3    | 0.00%  |
```

> 本机单实例结果，非集群能力上限。历史报告中另有一次 50 并发、27081 请求、1354 QPS 的完整压测，两者均错误率 0%，可互相印证。

9.2 登录限流

```
连续 14 次错误登录的状态码：
401 401 401 401 401 401 401 401 401 401 429 429 429 429
```

阈值 10 次/分钟/IP，第 11 次起返回 429，与设计一致。**限流不是把正常用户挡在门外**：校园网 NAT 下多人共享出口 IP，因此阈值取相对宽松值，另由密码错误计数兜底。

十、安全测试

检查项	方法	修复前	修复后
敏感端点匿名访问	GET /druid/datasource.json	匿名返回连接串与用户名	默认关闭，404
JWT 密钥管理	检查配置	硬编码默认值	支持 CAMPUS_JWT_SECRET 覆盖
跨域策略	带伪造 Origin 请求	反射任意 Origin	仅放行白名单
HTTP 状态码语义	错方法 / 错参数	200 + code 500	405 / 400
错误响应契约	非法请求体	problem+json	统一 {code,message,data}
前端安全响应头	检查 nginx 配置	缺少 CSP	CSP / Referrer-Policy 齐全

10.1 已验证的防护

- 认证：无令牌 / 伪造令牌 / 过期令牌均 401，不进入业务逻辑。
- 授权：越权按 404 处理，不泄露资源是否存在；查询按用户维度过滤。
- 注入：SQL 全部走 MyBatis 参数绑定；XSS 关键字与注入关键字均返回空结果，无 500。
- 幂等：唯一键兜底，重复提交不产生第二单。
- 密码：BCrypt 存储，错误提示统一为"手机号或密码错误"，不区分账号是否存在。
- 密钥：数据库口令与支付回调密钥均通过环境变量注入，仓库无明文；JWT 密钥支持 `CAMPUS_JWT_SECRET` 覆盖，保留的开发默认值仅用于本机，部署必须覆盖。

十一、本轮发现并修复的缺陷

#	缺陷	等级	根因	修复
1	Druid 控制台匿名可访问，/druid/datasource.json 直接返回 JDBC 连接串与用户名	高	statViewServlet 默认 enabled: true 且 allow: 留空即允许所有 IP	默认关闭；需要时 CAMPUS_DRUID_ENABLED=true 开启，且默认只允许本机
2	JWT 签名密钥硬编码为默认字符串	高	配置直接写死，未走环境变量	改为 ${CAMPUS_JWT_SECRET:默认值}，部署必须覆盖
3	CORS 反射任意 Origin	中	addAllowedOriginPattern("*")	改为白名单 campus.cors.allowed-origins，默认仅本地 Vite
4	方法不支持返回 HTTP 200 + body code 500	中	异常发生在 handler 解析阶段，落到若依全局处理器并被包成 200	新增作用于 /api/v1 的 MVC 异常 advice，返回 405
5	参数类型错误返回 HTTP 200 + code 500	中	同上	返回 400，避免客户端错误污染 5xx 指标
6	校验失败返回 problem+json，破坏前端契约	中	新增 advice 优先级高于 campus 包内处理器，父类兜底渲染	在 advice 内重写校验/不可读异常，统一为 {code,message,data}

> **修复的验证方式：**新增 ApiMvcExceptionAdviceTest（5 个用例）锁定 405/400/统一契约/非 API 路径回退；接口脚本 34 项全部复测通过；后端 44 项回归通过。**缺陷不是"改完就算"，而是用测试把它钉住。**

十二、持续集成

作业	状态	关键步骤
后端编译与测试	通过	JDK 17 → 44 用例 → OpenAPI 契约漂移校验 → 覆盖率门禁（≥35%）→ 上传产物
前端构建与测试	通过	Node 20 → 19 用例 → 构建 → 依赖漏洞门禁（生产依赖 high/critical）
容器编排验证	通过	docker compose 起四服务 → 健康检查 → 冒烟

最近一次：CI #47（提交 87c6aa7）三个作业全部 success，总耗时 259 秒。此前 #43 因 YAML 缩进损坏、#44 因配置问题失败，均已定位修复——这也说明 CI 是真在跑，不是摆设。



图 1　GitHub Actions：三个作业的运行与结果

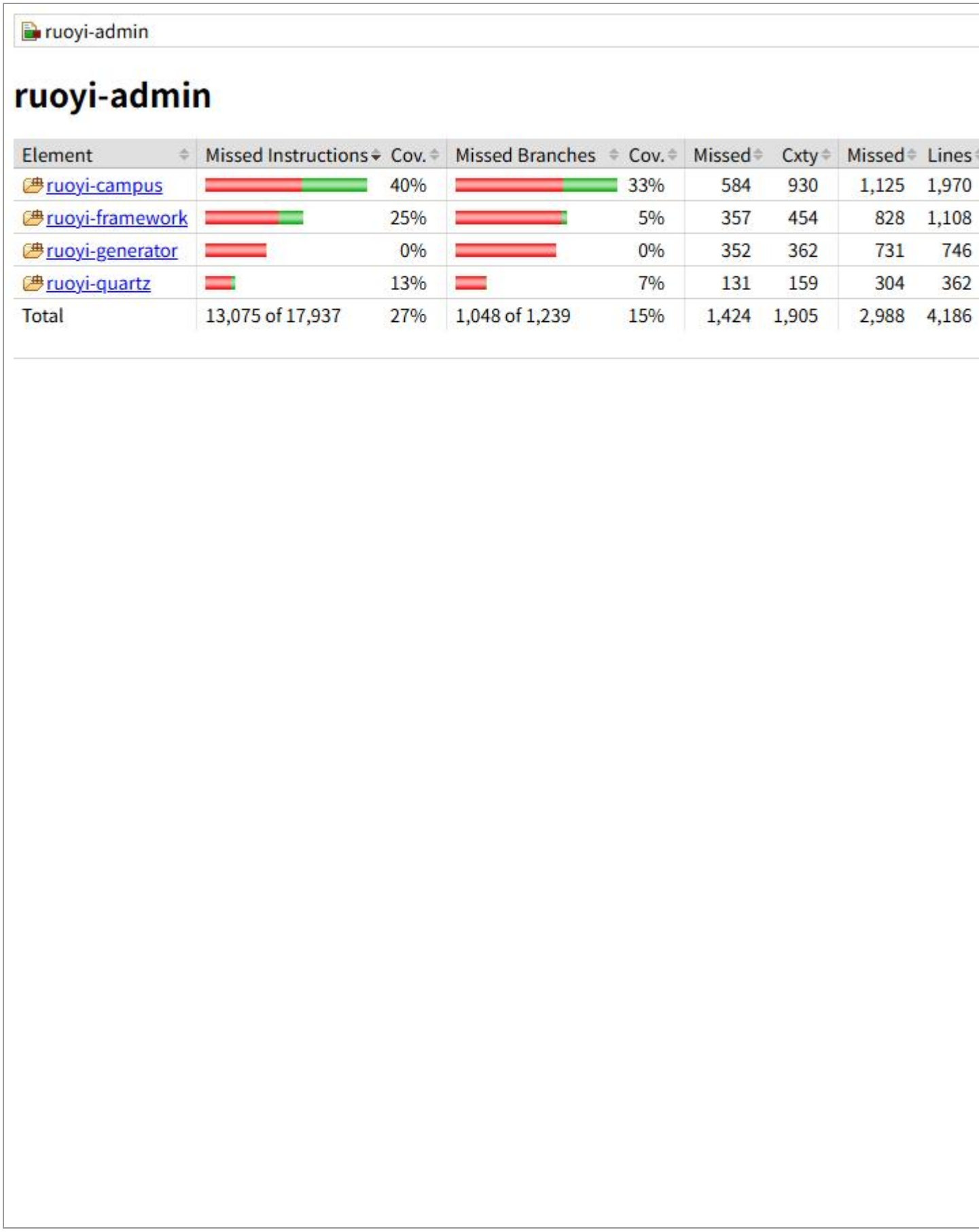


图 2　JaCoCo 覆盖率报告

GET /actuator/prometheus —— 指标暴露

```
application_ready_time_seconds{application="campus-shop"
disk_free_bytes{application="campus-shop",path="C:\\User
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_sum{application="campus-
shop",error="none",exception="none",method="GET",outcome
http_server_requests_seconds_count{application="campus-
```

图 3　Prometheus 指标端点

十三、遗留问题与风险

#	遗留项	影响	说明与触发条件
R1	MySQL 单实例	中	数据库宕机则整体不可用；主从开关已预留，触发条件为写 QPS 持续 > 200
R2	无 E2E 自动化（Playwright）	中	以 compose 冒烟 + 演示脚本替代；页面级回归依赖人工
R3	无静态检查（Checkstyle/ESLint）	低	以编译 + 契约校验兜底；代码风格靠约定
R4	覆盖率 42.2%，未达 70%	低	Controller 层薄，主要缺的是异常分支；有 35% 门禁防退化
R5	未做第三方渗透测试	中	已做 OWASP Top10 自查与接口级探针，但不等于专业渗透结论
R6	缺少长时间稳定性测试	低	当前压测 10 秒；建议上线前补 2 小时 soak

以上均记录在 `docs\风险登记表.md`，并写明触发条件与处置方案，不是"忘了做"。

十四、结论

本次测试覆盖了功能、安全、性能、端到端四条主线，共执行 **97 个自动化用例**（44 后端 + 19 前端 + 34 接口）和 **22 项端到端与专项检查**，全部通过。

测试过程中发现 **6 项缺陷**，其中 2 项为高危安全问题（Druid 控制台匿名可访问、JWT 密钥硬编码）。所有缺陷均已修复，并通过新增用例与全量回归验证。

系统在功能正确性、并发安全（防超卖 / 防重复下单）、权限隔离、可观测性方面达到可交付水平；性能在单实例 20 并发下为 1048 QPS、错误率 0%。

剩余风险集中在"高可用与工程化工具链"（数据库主从、E2E、静态检查、渗透测试），均已知、已评估、已写明触发条件，不影响本次交付判断。

> **结论：测试通过。**系统满足校园二手交易场景的功能与安全要求，可以交付；上线前建议优先处理 R1（数据库主从）与 R5（渗透测试）。